Trabalho integrador — O repertório da turma

Versão 3 · 10/11/2026 · Valor: 4,0 pontos. Substitui a versão 2, de 27/10/2026.
IFSul — Campus Passo Fundo · Ensino Médio Integrado — Técnico em Informática
Disciplina: Programação III
Professor: Élder F. F. Bernardi
Tutor da atividade: Prof. Leonardo Constantin
Turmas: 3D1 e 3D2 · **Modalidade:** duplas, com verificação individual
Apresentações: 3D2 em 24/11/2026; 3D1 em 04/12/2026
Tempo: quatro encontros de 150min + até 60min extraclasse por encontro; apresentação em outro encontro de 150min.

Todos os materiais do trabalho estão na página do trabalho e no Classroom da disciplina.

> **O que mudou na versão 3**
>
> - **Nenhuma regra mudou.** Requisitos R1 a R8, marcos, datas, critérios, pesos, níveis, entrega, uso de IA e recuperação são os mesmos da versão 2.
> - Quem já começou o trabalho continua do ponto em que está. Não é preciso refazer nada.
> - O texto foi reorganizado: há um resumo no início, as instruções estão em listas e os materiais de apoio têm link direto.
> - As seções da versão 2 continuam com os mesmos títulos e na mesma ordem.
> - Seções novas: "Como usar este documento", "Resumo do trabalho" e "Glossário".

Sumário

1. Como usar este documento
2. Resumo do trabalho
3. O que vocês vão resolver
4. De onde partir
5. Como as partes se conectam
6. Banco de dados: preparar e conferir
7. Núcleo do trabalho — R1 a R8
8. Quatro encontros para desenvolver
9. Percurso para testar e demonstrar
10. O que guardar e entregar
11. O que cada integrante precisa compreender
12. Como será avaliado
13. Uso de IA
14. Correção e recuperação
15. Aprofundamentos por escolha
16. Se houver impedimento técnico
17. Apoios da disciplina
18. Glossário

Como usar este documento

1. Leiam primeiro o **Resumo do trabalho**. Ele é curto.
2. Depois, leiam a seção **Quatro encontros para desenvolver**. Ela diz o que mostrar em cada encontro.
3. As outras seções são de consulta. Vocês podem ler cada uma quando precisarem dela.

Neste documento:

- **Núcleo** é a parte obrigatória do trabalho: os requisitos R1 a R8.
- **Opcional** é o que a dupla escolhe fazer ou não. O que é opcional não é exigido para a nota máxima e não reduz a nota.

- As tabelas de requisitos, marcos, critérios e níveis têm o mesmo texto das versões anteriores.
- As explicações novas desta versão não criam regra. As explicações novas são: o resumo, as listas que vêm depois das tabelas, as linhas "Deu certo se" e o glossário.
- Se vocês notarem diferença entre uma explicação e uma tabela, vale a tabela. Nesse caso, avisem o professor.
- As palavras técnicas estão explicadas no **Glossário**, no fim do documento.

Ficaram com dúvida sobre o que fazer? Perguntem ao professor.

↑ Voltar ao sumário

Resumo do trabalho

Pergunta	Resposta
O que é o trabalho?	Um karaokê na web, com um catálogo de músicas. Pela tela, quem usa o karaokê consulta, cadastra, edita e exclui músicas. Os dados ficam em um banco de dados. A aplicação fica publicada em uma URL pública.
O que é obrigatório?	O núcleo: os requisitos R1 a R8.
Com quem?	Em dupla. Cada pessoa da dupla também passa por uma verificação individual.
Quanto vale?	4,0 pontos. Até 2,8 pontos vêm da dupla. Até 1,2 ponto vem da compreensão de cada pessoa.
Quando?	Quatro encontros para desenvolver e um encontro para apresentar. As datas estão na seção "Quatro encontros para desenvolver".
O que entregar?	O link do repositório GitHub, a URL pública da aplicação e o caminho do README.
Onde entregar?	No Classroom da disciplina.

O caminho do trabalho, em ordem:

1. **M1:** abrir a página, listar as músicas vindas da API, conferir o banco e iniciar a publicação.
2. **M2:** cadastrar uma música pela tela e conferir que ela continua no banco depois de reiniciar o servidor.
3. **M3:** editar e excluir pela tela, validar campos vazios, mostrar mensagens de erro e repetir o CRUD na URL pública.
4. **M4:** conferir tudo na URL pública, completar o README e ensaiar.
5. **Apresentação:** demonstrar o trabalho. Cada pessoa mostra o que compreendeu.

Esta lista é um resumo. A lista completa de cada marco está na seção "Quatro encontros para desenvolver".

↑ Voltar ao sumário

O que vocês vão resolver

A turma está organizando um repertório para um karaokê. Repertório é o conjunto de músicas do catálogo.

Duas situações precisam funcionar:

- **Consultar o catálogo.** Quem usa o karaokê encontra uma música, confere o artista e reproduz as partes disponíveis.
- **Manter o repertório.** Quem mantém o repertório inclui uma música, corrige os dados de uma música ou retira um registro. Tudo isso é feito pela própria tela, sem escrever SQL para cada alteração.

O desafio é completar **interface → API → DAO → banco → resposta na interface**, com a aplicação acessível por uma URL pública. Os dados precisam continuar disponíveis depois de reiniciar o servidor. Usem dados de demonstração. O trabalho não exige login.

Este trabalho continua o karaokê que vocês exploraram no terminal, na API, no player e na integração com banco. Cada projeto pode estar numa etapa diferente. Vocês podem começar de onde o projeto de vocês está.

A primeira meta é **abrir a página e listar músicas vindas da API**. Completem uma operação por vez.

↑ Voltar ao sumário

De onde partir

Vocês podem partir de um destes dois pontos:

Ponto de partida	O que é
Projeto da dupla	O karaokê que vocês já vinham fazendo nas aulas. Vocês aproveitam o que já funciona.
Base de retomada	Um projeto preparado pelo professor. Ele está no Classroom e em base_retomada.zip, na página do trabalho.

A base é opcional, não reduz a nota e permite alcançar o nível excelente.

- A base vem acompanhada do Roteiro de retomada.
- Vocês podem recorrer à base durante o trabalho. Se fizerem isso, preservem os registros úteis que já existem.
- Os requisitos são os mesmos nos dois pontos de partida. Quem usa projeto próprio aproveita o que já funciona e demonstra os mesmos requisitos.
- Em dúvida sobre qual escolher? O Guia do dia 1, no passo 2, ajuda a decidir.

O que a base já traz pronto: leitura, listagem, seleção, player, API e DAOs.

O que a dupla faz na base: três tarefas de integração, no arquivo `public/admin.js`:

1. Conectar o formulário ao **cadastro** de música.
2. Aproveitar a seleção para **editar** a música escolhida.
3. Conectar a ação de **exclusão**, com confirmação.

Em cada uma das três tarefas:

- tratem a resposta;
- informem sucesso ou erro;
- atualizem a lista.

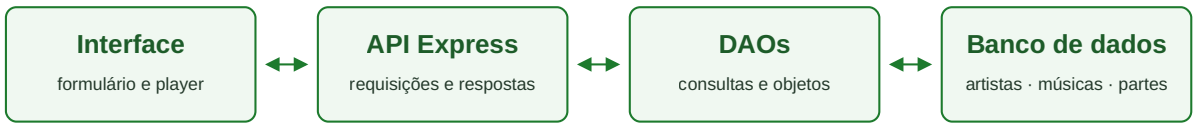
Os controles de escrita começam indisponíveis: habilitem cada ação quando sua integração estiver pronta. Na base, os controles de escrita são os botões de salvar, de excluir e de nova música. No arquivo `public/admin.js`, as três tarefas estão marcadas como `TODO 1`, `TODO 2` e `TODO 3`.

Tecnologias:

- HTML, CSS e JavaScript com `fetch` atendem ao núcleo. Vue é opcional.
- Reaproveitem as classes, os DAOs e as relações que já existem.
- O núcleo não exige trocar a arquitetura nem adicionar dependências.
- O Roteiro de retomada informa como instalar e executar a base com Node.js 22 ou superior.

↑ Voltar ao sumário

Como as partes se conectam



O retorno percorre as mesmas camadas até atualizar a interface.

O caminho dos dados, na ordem:

1. Alguém usa a **interface**: a página aberta no navegador.
2. A interface envia uma requisição para a **API**, com `fetch`.
3. A API chama o **DAO**.
4. O DAO lê ou altera o **banco de dados**.
5. A resposta volta pelo mesmo caminho. A interface mostra o resultado.

Os dados:

- Um artista pode ter várias músicas; uma música pode ter várias partes.
- O CRUD obrigatório na interface é o de **músicas, com nome e artista**.
- As outras tabelas não precisam de telas próprias de administração.

O formato das requisições:

- Na base, página e API usam a mesma origem: a interface chama `/api/musicas`. Por isso, não é preciso configurar CORS.
- No POST e no PUT, enviem `nome` e `artista` como **strings**.
- Na resposta, `artista` é um **objeto** com `id` e `nome`.
- Para preencher o formulário, usem o nome contido nesse objeto.

Editar o artista de uma música significa alterar o nome informado e o vínculo daquela música com o artista correspondente. A mudança vale só para aquela música: as outras músicas do catálogo continuam com o artista que já tinham.

↑ Voltar ao sumário

Banco de dados: preparar e conferir

Esta seção orienta o preparo do banco; os requisitos continuam sendo R1 a R8. Os guias da disciplina usam um projeto Supabase por dupla, com dados de demonstração.

Situações mais comuns:

Situação da dupla	O que fazer
Vocês já têm o projeto Supabase da Unidade 3	Podem continuar com ele. Antes, recomendamos rodar o diagnóstico e os ajustes que ele indicar.
Vocês vão criar um projeto novo	O passo a passo de criação está no guia.

Os dois casos estão na etapa 4 do Guia passo a passo do Supabase.

O diagnóstico confere se as tabelas estão como a aplicação espera. Por exemplo: ele confere se excluir uma música também remove as partes dela.

Onde ficam as credenciais:

As credenciais, como a chave secreta do Supabase, ficam somente no servidor:

- no `.env` local;
- nas variáveis de ambiente do serviço publicado.

Não as coloquem no JavaScript público, no Git, em capturas ou no README.

Projeto pausado:

O plano gratuito do Supabase pausa projetos sem atividade por 7 dias. Com o projeto pausado, o banco não responde e a aplicação publicada passa a dar erro.

Antes da apresentação:

1. Abram o painel do projeto no Supabase.
2. Confiram o status.
3. Se o projeto estiver pausado, restaurem o projeto (botão **Restore**) e esperem o banco voltar a responder.
4. Se o projeto não voltar a responder, sigam a seção "Se houver impedimento técnico".

↑ Voltar ao sumário

Núcleo do trabalho — R1 a R8

O núcleo é a parte obrigatória. Ele tem oito requisitos:

ID	Resultado esperado
R1 — Consultar	Listar músicas da API e selecionar uma para consultar seus dados, mostrando nome e artista.

ID	Resultado esperado
R2 — Cadastrar	Enviar nome e artista pela interface; o registro criado aparece em nova consulta e tem ID identificável na resposta da API.
R3 — Editar	Alterar nome/artista da música escolhida, mantendo seu ID e conservando a mudança no banco.
R4 — Excluir	Identificar a música, pedir confirmação e remover pela interface; cancelar mantém o registro.
R5 — Validar e informar	Campos obrigatórios vazios não geram cadastro válido; mensagens distinguem sucesso de erro, inclusive falha de conexão.
R6 — Preservar o repertório	Reproduzir uma música com partes do repertório inicial; música nova sem partes é válida e mostra a situação com Tocar indisponível.
R7 — Persistir e publicar	Executar o CRUD na URL pública com banco; conferir os dados após reiniciar o servidor, antes de excluir o registro de teste.
R8 — Explicar e documentar	Cada pessoa acompanha um fluxo e realiza ou prevê uma pequena intervenção; a dupla registra execução, testes e uma decisão no README.

Onde conferir cada requisito:

Requisito	Onde está o teste ou a explicação
R1	Passo 1 do "Percurso para testar e demonstrar".
R2	Passo 2 do Percurso.
R3	Passo 3 do Percurso.
R4	Passos 5 e 6 do Percurso.
R5	Passo 7 do Percurso e "Falha controlada local".
R6	Passos 1 e 2 do Percurso.
R7	Passo 4 do Percurso. O CRUD também é executado na URL pública (marco M3).
R8	Seções "O que cada integrante precisa compreender" e "O que guardar e entregar".

Detalhes do núcleo:

- O ID pode ficar fora do layout: confiram-no na resposta da API, na aba Rede do navegador, com orientação do professor.
- Música sem partes continua listável, editável e removível.
- Cadastrar letras não faz parte do núcleo.
- Preservem ao menos uma música do repertório inicial para o player.
- Para a tela, bastam campos identificados, texto legível e ações utilizáveis.

Ficam fora do núcleo: relatório de dados, técnica específica de responsividade, fila, upload, áudio, MIDI, sincronização e autenticação.

↑ Voltar ao sumário

Quatro encontros para desenvolver

O trabalho essencial acontece em aula, incluindo a configuração inicial e o apoio para a integração.

O tempo:

- Para cada encontro, são **150min em aula + até 60min extraclasse** para revisão, testes e documentação.
- No total, são **600 + até 240min**, ou seja, até **14h** de desenvolvimento.
- A apresentação final ocupa mais um encontro de 150min.

Os marcos e as datas:

Marco	3D2 — 2026	3D1 — 2026	O que mostrar ao final
M1 — Abrir e compreender	27/10	30/10	Lista da API, rota/DAO e situação do banco identificados; escolher base, iniciar publicação e registrar URL inicial ou impedimento.
M2 — Escrever e conservar	03/11	06/11	Cadastro pela tela, ID e consulta após reinício. Meta: listagem/cadastro publicados com banco. Iniciar observações individuais.
M3 — Fechar o CRUD	10/11	13/11	Edição, exclusão, validação/erro, música sem partes e player conferidos; repetir CRUD na URL pública.
M4 — Consolidar e ensaiar	17/11	27/11	Conferência completa, persistência pública, README e observações individuais; corrigir pendências.
Apresentação	**24/11**	**04/12**	Percurso integrado e evidências de compreensão dos dois integrantes.

Os marcos orientam o acompanhamento e a retomada. Não há desconto automático por marco.

As listas abaixo explicam a coluna "O que mostrar ao final", um item por linha.

M1 — o que mostrar

- [ ] A lista de músicas vinda da API.
- [ ] Onde ficam a rota e o DAO no código.
- [ ] A situação do banco. Por exemplo: funcionando, pausado ou ainda não criado.
- [ ] O ponto de partida escolhido: projeto da dupla ou base de retomada.
- [ ] A publicação iniciada.
- [ ] A URL inicial registrada, ou o impedimento registrado.

Extraclasse proposta, até uma hora: revisar o mapa dos arquivos e as instruções de execução.

Apoio: Guia do dia 1 — Repositório, Supabase e Render.

M2 — o que mostrar

- [ ] O cadastro de uma música pela tela.
- [ ] O ID da música cadastrada.
- [ ] A consulta da mesma música depois de reiniciar o servidor.
- [ ] Meta: listagem e cadastro publicados, com banco.

Neste encontro começam as observações individuais.

Extraclasse proposta, até uma hora: repetir cadastro e reconsulta e registrar o resultado.

Apoio: Guia de interface — CRUD na tela com fetch.

M3 — o que mostrar

Para conferir cada item, façam o passo indicado do "Percurso para testar e demonstrar".

- [ ] A edição conferida: Passo 3 do Percurso.
- [ ] A exclusão conferida: Passos 5 e 6 do Percurso.
- [ ] A validação e as mensagens de erro conferidas: Passo 7 do Percurso e "Falha controlada local".
- [ ] A música sem partes conferida: Passo 2 do Percurso.
- [ ] O player conferido: Passo 1 do Percurso.
- [ ] O CRUD repetido na URL pública.

Extraclasse proposta, até uma hora: conferir o checklist e preparar a explicação de um fluxo.

Apoio: seção "Percurso para testar e demonstrar", neste documento, e Guia de interface — CRUD na tela com fetch.

M4 — o que mostrar

- [ ] A conferência completa.
- [ ] A persistência na URL pública.
- [ ] O README.
- [ ] As observações individuais.
- [ ] As pendências corrigidas.

Extraclasse proposta, até uma hora: revisar o README, ensaiar a demonstração e organizar as evidências.

Apoio: Guia de deploy na Render, seções "Antes da apresentação" e "Checklist final de entrega".

Apresentação — o que mostrar

- [ ] O percurso integrado.
- [ ] As evidências de compreensão dos dois integrantes.

Sobre o trabalho extraclasse e os imprevistos

- As tarefas extraclasse não pedem funcionalidades novas.
- Se faltar acesso fora da aula, registrem a dificuldade para que o apoio seja organizado.
- Se algo não funcionar, localizem o primeiro ponto do percurso que falha. Retomem esse ponto com o professor.

↑ Voltar ao sumário

Percurso para testar e demonstrar

Executem esta sequência no projeto concluído, na ordem dos passos.

Antes de começar:

- Nas escritas (cadastrar, editar e excluir), usem uma música nova, sem partes. Neste documento, ela se chama **música de teste**.
- Preservem a música do repertório inicial que vocês usam para demonstrar o player.
- Confiram o dado antes de removê-lo.

Passo 1 — Consultar e reproduzir

1. Abram a lista.
2. Selecionem uma música do repertório inicial.
3. Demonstrem o player com as partes dessa música.

Deu certo se: a tela mostra o nome e o artista da música, e o player reproduz as partes.

Passo 2 — Cadastrar

1. Criem pela tela uma música de teste com nome e artista, sem partes.
2. Confiram a resposta da API, na aba Rede do navegador. O Guia de interface explica como, na seção Conferir na aba Rede.
3. Anotem o ID.
4. Consultem a música novamente.
5. Selecionem a música de teste.

Deu certo se:

- a resposta da API mostra o ID da música criada;
- a música aparece na nova consulta;
- a tela informa que não há partes e mantém Tocar indisponível.

Passo 3 — Editar

1. Mudem nome/artista da música de teste.
2. Confiram a atualização na lista.
3. Confiram a atualização na resposta da API.

Deu certo se:

- a lista mostra a atualização;
- a resposta da API mostra a atualização;
- o ID continua o mesmo.

Passo 4 — Conferir persistência

Façam este passo antes de excluir a música de teste.

1. Reiniciem o servidor.
2. Consultem novamente.
3. Mostrem o mesmo ID com os dados alterados.
4. Repitam a conferência no serviço publicado, com orientação para o reinício. O passo a passo está na Etapa 8.4 do Guia de deploy na Render.

Deu certo se: depois do reinício, a consulta mostra o mesmo ID com os dados alterados. Isso vale para o servidor local e para o serviço publicado.

Passo 5 — Cancelar exclusão

1. Iniciem a exclusão da mesma música de teste sem partes.
2. Cancelem.
3. Façam uma nova consulta.

Deu certo se: a nova consulta mostra que a música continua cadastrada.

Passo 6 — Confirmar exclusão

1. Iniciem a exclusão da música de teste outra vez.
2. Confirmem a remoção.
3. Consultem novamente.

Deu certo se: o registro não aparece mais.

Preservem a música usada para demonstrar o player.

Passo 7 — Conferir validação

1. Tentem cadastrar com nome vazio.
2. Tentem cadastrar com artista vazio.

Deu certo se: nas duas tentativas, a interface informa o problema, sem anunciar um cadastro válido.

Falha controlada local

Realizem este teste com orientação do professor.

1. Deixem a página aberta no navegador.
2. Parem somente o processo do servidor local.
3. Tentem uma requisição, como atualizar a lista ou cadastrar uma música de teste.
4. Observem a mensagem de erro: a interface não pode afirmar sucesso.
5. Reiniciem o processo.
6. Consultem a lista para conferir o estado.

Deu certo se: com o servidor parado, a interface mostra uma mensagem de erro e não mostra mensagem de sucesso.

Neste teste:

- não alterem credenciais reais para simular falhas;
- não parem o serviço publicado.

↑ Voltar ao sumário

O que guardar e entregar

O que a dupla precisa ter:

- Código-fonte versionado, identificando os dois integrantes.
- URL pública com banco.
- README curto.
- Demonstração do núcleo e verificação individual no próprio projeto.

O que o README precisa ter:

- instalação e execução;
- nomes das variáveis necessárias, **sem valores secretos**;
- URL;
- uma decisão da dupla;
- um registro por marco, com data, teste, resultado e próximo passo.

Entrega pelo Classroom da disciplina:

1. Os dois integrantes devem constar na entrega.
2. Informem o link do repositório GitHub.
3. Informem a URL pública da aplicação.
4. Informem o caminho do README.

Mantenham o código e os registros no repositório; não é necessário anexar `node_modules/` ou um ZIP do projeto.

O que não é necessário: vídeo, relatório técnico separado e slides. Registro textual curto e demonstração observada bastam para acompanhar os testes.

O que não enviar: `.env`, chaves ou senhas.

A Ficha de marcos é um apoio opcional para organizar esses registros.

↑ Voltar ao sumário

O que cada integrante precisa compreender

Esta seção explica a verificação individual. Ela vale 30% da nota (critério "Compreensão — individual").

Durante os encontros:

- Alternem quem conduz e quem revisa.
- Ambos devem passar pela interface e pela API/DAO.

As observações individuais:

- Cada pessoa tem duas observações individuais.
- Elas podem ocorrer nos encontros 2–4 e ser complementadas na apresentação. Isso evita concentrar tudo no último dia.
- As tarefas terão objetivos e dificuldade comparáveis para os dois integrantes.

O que cada pessoa faz, ao longo das observações:

1. Acompanha um caminho dos dados.
2. Realiza ou prevê uma pequena mudança já exemplificada.

Prever uma mudança é dizer o que aconteceria se ela fosse feita.

As mudanças já exemplificadas são:

- validar nome vazio ou artista vazio;
- tratar retorno de erro;
- conferir persistência após reinício;
- explicar o vínculo entre artista, música e partes.

Como vocês podem responder:

- Vocês podem consultar o próprio código e os materiais de apoio.
- Vocês podem explicar oralmente, apontar o percurso no código ou usar anotação ou desenho com demonstração.

O que não é critério: digitação rápida, memorização e fluência na fala.

↑ Voltar ao sumário

Como será avaliado

Dimensão	Evidência observada	Peso	Máximo
Integração e funcionamento — dupla	CRUD pela tela, dados coerentes e player preservado.	30 %	1,2 ponto
Persistência e publicação — dupla	Banco e dados conservados após reinício; operações na URL pública.	20 %	0,8 ponto
Qualidade proporcional ao ensino — dupla	Validação, feedback útil e responsabilidades identificáveis.	10 %	0,4 ponto
Compreensão — individual	Cada pessoa segue o fluxo e demonstra uma intervenção equivalente.	30 %	1,2 ponto
Processo e comunicação — dupla, com participação de ambos	README reproduzível, testes e decisões registrados.	10 %	0,4 ponto

Os quatro níveis de desempenho estão na Rubrica dos estudantes.

O que vale para a nota:

- **O núcleo excelente e compreendido permite nota máxima, sem funcionalidades extras.**
- A avaliação soma **2,8 pontos da dupla (70%)** e **1,2 ponto da compreensão de cada pessoa (30%)**.
- Cada integrante pode ter uma pontuação individual diferente.

O que não determina a nota: a origem da base, a quantidade de código/commits e o uso de Vue.

Como a pontuação é calculada:

Em cada critério, aplica-se o nível descrito na rubrica ao seu valor máximo:

Nível	Parcela do valor do critério
Excelente	100%
Bom	75%
Regular	50%
Insuficiente	25%

Exemplo: nível Bom em integração corresponde a `1,2 × 0,75 = 0,9 ponto`.

Evidência ausente ou impedida fica registrada para retomada. Ela não será confundida automaticamente com domínio insuficiente.

↑ Voltar ao sumário

Uso de IA

É permitido usar IA para compreender erros, discutir soluções, revisar código e apoiar a implementação.

O que registrar no README, de forma breve:

- a ferramenta utilizada;
- em que parte ajudou;
- como vocês conferiram o resultado.

Não é necessário transcrever conversas inteiras.

Responsabilidade:

- Cada integrante continua responsável por compreender e testar o código entregue.
- O uso declarado de IA não reduz a nota.
- A compreensão será observada no projeto, nos marcos e na verificação individual.

Recomendação: usem o `AGENTS.md` fornecido. Ele orienta o agente de IA a explicar e revisar sem entregar a solução pronta.

- Onde está: no repositório da aula, na base de retomada e em AGENTS.md, na página do trabalho.
- Onde está explicado: na etapa 0 do Guia passo a passo do Supabase.

↑ Voltar ao sumário

Correção e recuperação

Durante os marcos: corrijam as pendências no mesmo repositório, preservando os registros anteriores.

Na recuperação:

- Cada dupla ou estudante poderá corrigir os pontos não demonstrados e produzir nova evidência equivalente.
- Isso acontece com consulta aos materiais e orientação do professor.
- A nova conferência usará os mesmos critérios e pesos.
- O professor comunicará o momento da retomada das pendências após o acompanhamento.
- Funcionalidades extras não serão condição de recuperação.

↑ Voltar ao sumário

Aprofundamentos por escolha

Esta seção é opcional. A nota máxima não depende dessas escolhas, e elas não substituem a compreensão.

Assim que demonstrarem o núcleo funcionando, vocês podem escolher um aprofundamento, sem esperar o M4. Conversem com o professor sobre o recorte.

Problema	Conteúdos a explorar	Primeiro limite sugerido
Encontrar músicas numa lista maior.	Busca/filtro, eventos e apresentação dos dados.	Filtrar nome/artista, preservando o CRUD existente.

Problema	Conteúdos a explorar	Primeiro limite sugerido
Facilitar a entrada de partes de uma música.	Formulários, JSON, relação 1:N e ordenação.	Editor com cadastro/consulta pelas rotas existentes; sem áudio, MIDI ou sincronização fina.
Organizar pedidos de músicas.	Lista, estados e vínculo com música.	Uma fila simples, sem exigir atualização em tempo real.
Reorganizar a interface com Vue.	Componentes, reatividade e consumo da API.	Preservar o mesmo contrato e os mesmos testes do núcleo.

O núcleo não exige dependências novas.

↑ Voltar ao sumário

Se houver impedimento técnico

Impedimento técnico é um problema técnico que impede a dupla de demonstrar uma parte do trabalho. Exemplo: o projeto do Supabase não volta a responder.

O que fazer:

1. Registrem no README a data, a etapa, o erro (sem segredos) e as tentativas de solução.
2. Mostrem o que funciona localmente com banco.
3. Preservem as evidências anteriores.

O professor orientará a retomada da verificação impedida. A publicação com banco continua sendo objetivo, sem penalidade automática por indisponibilidade externa.

↑ Voltar ao sumário

Apoios da disciplina

Todos os materiais estão no Classroom da disciplina e na página do trabalho, em HTML, PDF e texto (`.md`).

A seção **Downloads** da página do trabalho tem os arquivos para baixar: os dois pacotes do Classroom, a base de retomada, o exemplo do mural, o `AGENTS.md` e os quatro arquivos SQL.

Qual material abrir:

Vocês precisam de	Material
Preparar o projeto, o banco e a publicação no primeiro encontro	Guia do dia 1 — Repositório, Supabase e Render
Ir do repositório da aula até a API gravando no banco, etapa por etapa	Guia passo a passo do Supabase. A etapa 0 explica o `AGENTS.md`; a etapa 4, o diagnóstico do banco.
Fazer a página cadastrar, editar e excluir pela API	Guia de interface — CRUD na tela com fetch. Traz um exemplo completo e a aplicação às músicas.
Publicar, configurar variáveis de ambiente e conferir o serviço publicado	Guia de deploy na Render
Instalar e executar a base de retomada e fazer as tarefas de integração	Roteiro de retomada

Vocês precisam de	Material
Ver os quatro níveis de desempenho de cada critério	Rubrica dos estudantes
Organizar os registros de cada marco (apoio opcional)	Ficha de marcos

Atividades anteriores que ajudam (no Classroom da disciplina):

- **Atividade "CRUD de Dispositivos com fetch"** (Atividade 3): eventos, requisições e atualização da tela.
- **Atividade "Evoluindo a API do Karaokê — DAO, Múltiplas Músicas e Player no Navegador"** (Atividade 5): rotas e responsabilidades.
- **Atividade "Karaokê com Banco de Dados — DAOs no Supabase"** (Unidade 3): relações, persistência e configuração do servidor.

↑ Voltar ao sumário

Glossário

As explicações abaixo valem para este trabalho.

Palavra	O que significa neste trabalho
API	O conjunto de rotas do servidor que a página chama, como `/api/musicas`.
Base de retomada	O projeto preparado pelo professor, de uso opcional. Também aparece como "base".
CRUD	As quatro operações com um registro: cadastrar, consultar, editar e excluir.
DAO	A classe que lê e altera os dados no banco.
Encontro	Uma aula de 150min do trabalho.
Evidência	O que mostra que algo funciona ou foi compreendido. Exemplos: uma demonstração observada, um registro curto de teste, uma explicação.
Fluxo ou **caminho dos dados**	O caminho interface → API → DAO → banco → resposta na interface.
ID	O número que identifica um registro.
Interface	A página que a pessoa usa no navegador. Também aparece como "tela".
Intervenção	Uma pequena mudança ou tarefa, de um dos tipos listados na seção "O que cada integrante precisa compreender".
Marco	O ponto de conferência de um encontro. São quatro: M1, M2, M3 e M4.
Música de teste	A música nova, sem partes, que vocês criam para testar. Ela é excluída no passo 6 do Percurso.
Núcleo	A parte obrigatória do trabalho: os requisitos R1 a R8.
Parte	Um trecho da letra de uma música. O player reproduz as partes em ordem.
Percurso	A sequência de passos da seção "Percurso para testar e demonstrar".

Palavra	O que significa neste trabalho
Persistir	Guardar os dados no banco. Dado persistido continua disponível depois de reiniciar o servidor.
Player	A parte da tela que apresenta as partes da letra, uma depois da outra, quando alguém clica em Tocar. Na base de retomada, o player apresenta texto, sem áudio.
Publicar	Colocar a aplicação em um serviço na internet, como a Render, para que ela abra pela URL pública. "Serviço publicado" é a aplicação funcionando nesse serviço.
README	O arquivo `README.md` do repositório, com as instruções e os registros da dupla.
Registro	Tem dois sentidos neste trabalho. (1) Uma linha no banco de dados, como uma música cadastrada. (2) Uma anotação curta da dupla, como o registro de um marco no README.
Repertório inicial	As músicas que já estão no banco antes dos testes de vocês. O arquivo `seed.sql` inclui a música de exemplo `Encontro da turma`, que tem partes.
Requisito	Um resultado que o trabalho precisa apresentar. São oito: R1 a R8.
Rota	Um endereço da API com um método, como `GET /api/musicas`.
Seed ou **amostra**	Nomes usados na Rubrica, no Roteiro e na Ficha para a música de exemplo incluída pelo arquivo `seed.sql`.
URL pública	O endereço da aplicação publicada na internet.
Verificação individual	A observação do que cada pessoa compreendeu, feita no próprio projeto.

↑ **Voltar ao sumário**